arXiv:2304.07735v3 [cs.CR] 31 Mar 2024

Permutation Equivariance of Transformers and Its Applications

Hengyuan Xu[†], Liyao Xiang[*†], Hangyu Ye[†], Dixi Yao[‡], Pengzhi Chu[†] and Baochun Li[‡]

[†]Shanghai Jiao Tong University
[‡]University of Toronto

Abstract

Revolutionizing the field of deep learning, Transformer-based models have achieved remarkable performance in many tasks. Recent research has recognized these models are robust to shuffling but are limited to inter-token permutation in the forward propagation. In this work, we propose our definition of permutation equivariance, a broader concept covering both inter- and intra- token permutation in the forward and backward propagation of neural networks. We rigorously proved that such permutation equivariance property can be satisfied on most vanilla Transformer-based models with almost no adaptation. We examine the property over a range of state-of-the-art models including ViT, Bert, GPT, and others, with experimental validations. Further, as a proof-of-concept, we explore how real-world applications including privacy-enhancing split learning, and model authorization, could exploit the permutation equivariance property, which implicates wider, intriguing application scenarios. The code is available at https://github.com/Doby-Xu/ST

1. Introduction

Originating as a tool in natural language processing, Transformer now has permeated various fields such as computer vision, multi-modal tasks, etc. Transformer model, introduced in [32], has revolutionized the way of approaching sequence-based tasks, and further demonstrates versatility and power in a diverse set of tasks with the development of Bert[6], GPT [3, 27], ViT [8], etc. Meanwhile, its intriguing property is discovered, e.g., the outputs being robust or invariant to token shuffling [17, 26]. However, previous works either show the property through empirical observation [26], or by modifying the original model structure [17], typically for input-order-independent tasks.

The shuffling invariance property of Transformer is widely recognized but not clearly understood by the community. ViT shows superior permutation robustness compared to CNN in patch shuffling where the spatial information is totally disrupted [26], indicating that self-attention is invariant to patch ordering. Similarly, [17] proposes Set Transformer, satisfying *permutation equi-/in-variance* — the output of the model should not change under any permutation of the elements in the input set. Their model inherits the network architectures proposed by [36] where each input element is first independently fed into a feed-forward neural network and then aggregated by a pooling operation.

As we found, the previous notion of *permutation equi-/in-variance* is quite limited. In this work, we propose our definition of *permutation equivariance* — meaning that the model trained over any inter- or intra-token permutation is equivalent to the model trained over normal inputs, in contrast to the inter-token permutation in previous works. Our definition essentially pushes one step forward in two-folds: first, the former definition works at a coarser granularity, i.e., tokens are exchanged and permuted while ours incorporates both inter- and intra-token shuffling. Second, our proposed property is stricter in the sense that, it not only requires output equivariance under input permutation in forward propagation (forwarding equivariance) but also demands model weights to be equivalently trained in the backward propagation (backprop equi-/in-variance). We show such backprop equi-/in-variance is closely related to forwarding equivariance by modeling the permutation as row/column shuffling in matrices.

Our findings inspire many potential applications in real-world scenarios. We present a simple, yet effective privacy-enhancing technique for feature offloading, based on the permutation equivariance property. An honest-but-curious cloud merely provides computational service while being blind to the training data, inference data, and the trained model. We show that our method remarkably improves the data utility-privacy tradeoff which is intensively investigated in privacy-preserving split learning [15, 28, 33].

[*]Corresponding author: xiangliyao08@sjtu.edu.cn. The research was supported in part by National Science and Technology Major Project 2021ZD0112801, NSF China (62272306, 62032020, 62136006).

Moreover, the trained model is protected from being effectively fine-tuned by unauthorized parties, yet with almost no impact on the model's performance.

Highlights of our contributions include: first, we reveal the permutation equivariance property in the forward and backward propagation of neural networks. Second, by analyzing the inner workings of the Transformer model, we prove that a wide range of models satisfy permutation equivariance. Third, as a proof-of-concept, we design a privacy-enhancing mechanism and a model authorization scheme to show the promising use of the property. A series of experiments demonstrate the superiority of our design to the state-of-the-art methods in the application scenes.

2. Related Works

2.1. Transformer

Dispensing with recurrence and convolutions, Transformer [32] shows superior performance solely on attention mechanisms in translation tasks, and soon becomes the de-facto standard for natural language processing (NLP). Models such as Bert [6], GPT [3, 27], and a myriad of Transformer-based counterparts [18, 20] have consistently achieved state-of-the-art results across a wide spectrum of tasks. Most recently, with Transformer models as the core, Large Language Models (LLMs) have achieved great success in recognizing, translating, predicting, or generating text or other contents.

Inspired by its success in NLP, Vision Transformer (ViT [8]) was designed and its performance exceeds the state-of-the-art convolutional networks on a variety of image tasks such as classification [31], object detection [19], and semantic segmentation [5]. Substantial efforts have also been devoted into designing powerful pre-trained models [2, 30, 37] with ViT-based backbone, leading to the emergence of a growing family of networks [1, 4, 5, 19].

2.2. Permutation Equi-/In-variance

Permutation invariance property refers to that the output value for a given set is the same regardless of the order of objects in the set [17, 36]. A closely related property permutation equivariance describes that for function f, any permutation $\boldsymbol{P}$ and input $\boldsymbol{X}$, $f(\boldsymbol{PX}) = \boldsymbol{P}f(\boldsymbol{X})$. Commonly, the permutation takes place at the token level and poses additional requirements on models. For example, Deep sets [36] derives the necessary and sufficient conditions for permutation invariance in deep models, i.e., the function can be decomposed in the form $\rho(\sum_{\boldsymbol{X}\in\mathcal{X}} \phi(\boldsymbol{X}))$ for transformations ϕ and ρ. Set transformer [17] further instantiates ϕ and ρ to an adapted encoder and a decoder, respectively. But their permutation invariance limits to the neural network forwarding (or inference), but neglects the invariance in the backward propagation (or training). In contrast, our work does not alter the existing Transformer network structure and illustrates permutation properties in both forward and backward propagations.

Table 1. The notations used.

Symbol	Description	Shape
$\boldsymbol{X}, \boldsymbol{Y}$	Data, label	Task dependent
F_1, F_2	Embedding, task head	Task dependent
$\boldsymbol{Z}$	Feature/ output of F_1	$\mathbb{R}^{n\times d}$
$\hat{\boldsymbol{Y}}$	Prediction/ output of F_2	Same shape w/ $\boldsymbol{Y}$
$\mathrm{Enc}(\cdot)$	Transformer encoder	$\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$
$\mathrm{T}(\cdot)$	Transformer backbone	$\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$
$\boldsymbol{P}_R$	Row permutation matrix	$\mathbb{R}^{n\times n}$
$\boldsymbol{P}_C$	Column permutation matrix	$\mathbb{R}^{d\times d}$
$\boldsymbol{W}$	Weights of Transformer	$\mathbb{R}^{t\times d}$

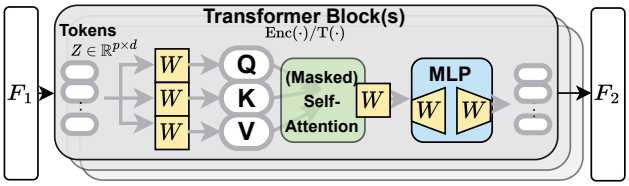
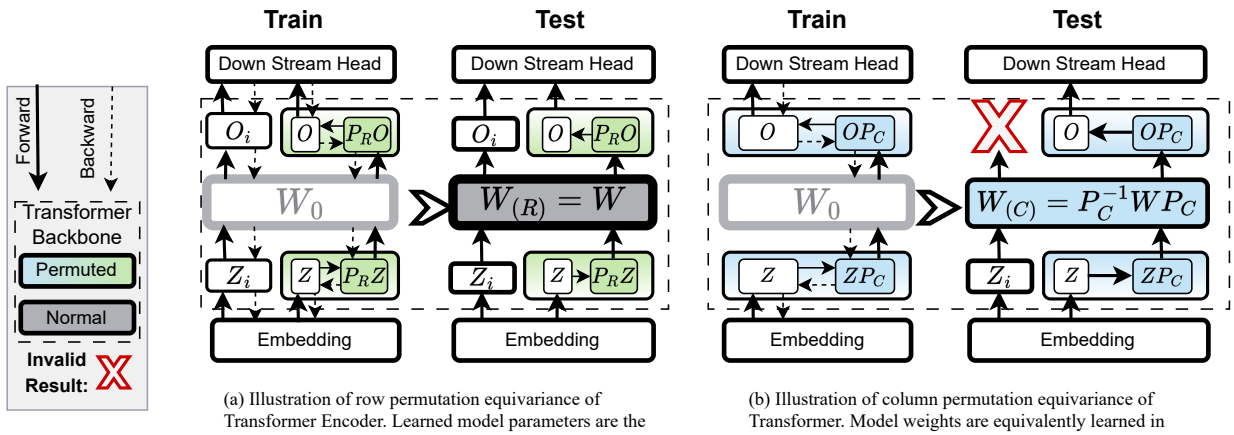


Figure 1. Illustration of Transformer backbone. Learnable weights in permutation are expressed by yellow blocks.

The (forwarding) permutation equivariance property has been widely recognized and applied in many works [9, 22, 26, 29, 33]. It is empirically verified by [26] that ViT is more robust to patch shuffling compared to convolutional networks. Leveraging such a property, Yao *et al.* propose a privacy-preserving split learning framework [33] by patch-shuffling the embedding. The permutation invariance property also sees applications in point cloud processing [9], reinforcement learning [29], neural architecture search [22], etc. However, most target tasks are input-order-insensitive (set-input problem). Different from them, our applications include tasks that rely on the order of the input (e.g., generating text).

3. Notations

We summarize the notations used in this paper in Tab. 1, where n is the number of tokens and d is the dimension of tokens. We use $\boldsymbol{W} \in \mathcal{R}^{t\times d}$ to generally denote the weight matrices of the Transformer backbone including weights in QKV projection, attention projection, and MLP. Details about the location of $\boldsymbol{W}$ in the structure of Transformer backbone are shown in Fig. 1. Transformer backbone usually consists of stacked encoders and decoders. Some components such as bias and LayerNorm do not have matrix-shaped weights, so we leave their discussion to Sec. 4.3.



(a) Illustration of row permutation equivariance of Transformer Encoder. Learned model parameters are the same, with or without permutation in training.

(b) Illustration of column permutation equivariance of Transformer. Model weights are equivalently learned in column permutation.

Figure 2. Illustration of permutation properties. $\boldsymbol{W}$ indicates main parameters in Transformer backbone (stacked Transformer encoders and decoders).

The matrix form of weights allows us to perform row or column permutations. We employ the subscript $_{(R)}$ to refer to row permutation of the corresponding features, weights, derivatives, etc., while using subscript $_{(C)}$ for column permutations. The subscript of $_{(P)}$ generally refers to row, column, and both row and column permutations. Specifically, the row permutation is equivalent to the inter-token permutation in previous works and column permutation is essentially intra-token shuffling. Row/column permutation is expressed by matrix multiplication with the permutation matrix: $\boldsymbol{P}_R\boldsymbol{Z}$, $\boldsymbol{Z}\boldsymbol{P}_C$, or $\boldsymbol{P}_R\boldsymbol{Z}\boldsymbol{P}_C$. The unshuffling is represented by multiplying the inverse of the permutation matrix, i.e., $\boldsymbol{P}_R^{-1}\boldsymbol{Z}_{(P)}$, $\boldsymbol{Z}_{(P)}\boldsymbol{P}_C^{-1}$, or $\boldsymbol{P}_R^{-1}\boldsymbol{Z}_{(P)}\boldsymbol{P}_C^{-1}$.

4. Properties and Proofs

We illustrate the key properties and provide their proofs in this section. Due to space limit, we collect most proofs in Appendix 9 and 10.

4.1. Permutation Equivariance of Transformer

In accordance with the inter-token permutation equivariance of the Transformer encoder [17], the row permutation equivariance can be expressed as

Theorem 4.1 (**Row Permutation Forward Equivariance**). *Transformer encoder is permutation equivariant w.r.t. token permutations, i.e. the row permutation of the input matrix, in forward propagation, i.e.,* $\mathrm{Enc}(\boldsymbol{P}_R\boldsymbol{Z}) = \boldsymbol{P}_R\mathrm{Enc}(\boldsymbol{Z})$ *for any permutation matrix* $\boldsymbol{P}_R \in \mathbb{R}^{n\times n}$.

Following the property, we perform row permutation to the output of the embedding layer F_1 and invert the permutation at the input to the downstream task head F_2, i.e., for

$$\boldsymbol{Z} = F_1(\boldsymbol{X}), \hat{\boldsymbol{Y}} = F_2(\mathrm{Enc}(\boldsymbol{Z})), \tag{1}$$

we perform shuffling and unshuffling as

$$\boldsymbol{Z}_{(R)} = \boldsymbol{P}_R F_1(\boldsymbol{X}), \hat{\boldsymbol{Y}}_{(R)} = F_2(\boldsymbol{P}_R^{-1}\mathrm{Enc}(\boldsymbol{Z}_{(R)})). \tag{2}$$

According to Thm. 4.1, the permuted input to F_2 is $\boldsymbol{P}_R^{-1}\mathrm{Enc}(\boldsymbol{P}_R\boldsymbol{Z}) = \mathrm{Enc}(\boldsymbol{Z})$. Given the shuffling and unshuffling operations, we can derive the following key property:

Theorem 4.2 (**Row Permutation Backward Invariance**). *In backward propagation, gradients of the Transformer encoder in the natural setting (Eq. 1) and the permuted setting (Eq. 2) are the same:*

$$\frac{\partial l}{\partial \boldsymbol{W}} = \frac{\partial l}{\partial \boldsymbol{W}_{(R)}}, \tag{3}$$

given loss function l.

With simple induction, we have:

Corollary 4.3. *The weights of the Transformer encoder learned in the natural setting (Eq. 1) and that learned in the permuted setting (Eq. 2) are the same:*

$$\boldsymbol{W} = \boldsymbol{W}_{(R)}. \tag{4}$$

The shuffling and inverse shuffling procedures and the properties are displayed in Fig. 2(a). In training, token embeddings can be randomly shuffled without affecting the learning results, provided that the input to the task head is inversely permuted. 'Without affecting' here means that the trained weights are exactly the same as the learned weights without any permutation.

Similar to row permutation, we could also perform column shuffling to the input, i.e., intra-token permutation:

$$\boldsymbol{Z}_{(C)} = F_1(\boldsymbol{X})\boldsymbol{P}_C, \hat{\boldsymbol{Y}}_{(C)} = F_2(\mathrm{T}_{(C)}(\boldsymbol{Z}_{(C)})\boldsymbol{P}_C^{-1}). \tag{5}$$